Appendix — Agent Authentication & Identity

> 🎯 **Goal**: Understand how AI agents authenticate, prove their identity, and securely access resources — inside Azure, outside Azure, in Copilot Studio, and via MCP tools.

> ⏱️ **Time**: 2–2.5h reading

> 📦 **Deliverable**: A clear mental model of which identity mechanism to use for each agent scenario, and how the wire-level auth flows work.

Table of Contents

1. Why Agent Identity Matters
2. Identity Mechanisms Overview
3. Inside Azure: Managed Identity
4. Inside Azure: Entra Agent ID
5. Azure AI Foundry Identity Model (Deep Dive)
6. Outside Azure: Service Principals & Federation
7. Outside Azure: Platform-Agnostic Standards
8. Copilot Studio Authentication
9. MCP Tool Authentication
10. Identity Proof Matrix
11. Interview Angle

Why Agent Identity Matters

AI agents are **non-human actors with real permissions**. They read databases, call APIs, modify files, send emails, and trigger workflows. Without proper identity:

- **No accountability** — who did what? Which agent called the API at 3am?
- **No access control** — an agent with blanket credentials can access everything
- **No revocation** — compromised agent credentials can't be selectively revoked
- **No compliance** — GDPR, HIPAA, SOX require auditable access trails

```
┌──────────────────────────────────────────────────────────────────┐
│                  THE AGENT IDENTITY PROBLEM                      │
```

```
┌──────────────────────────────────────────────────────────────────┐
│                                                                  │
│  Traditional world:     Human → Login → Token → Access resource  │
│                         (MFA, session, audit trail)              │
│                                                                  │
│  Agent world:           Agent → ??? → Token → Access resource    │
│                         No keyboard. No browser. No human present.│
│                                                                  │
│  Question: How does the agent prove "I am Agent-X and I'm allowed │
│            to do this" — without a human typing a password?      │
└──────────────────────────────────────────────────────────────────┘
```

The answer varies by **where the agent runs** and **what it needs to access**:

Identity Mechanisms Overview

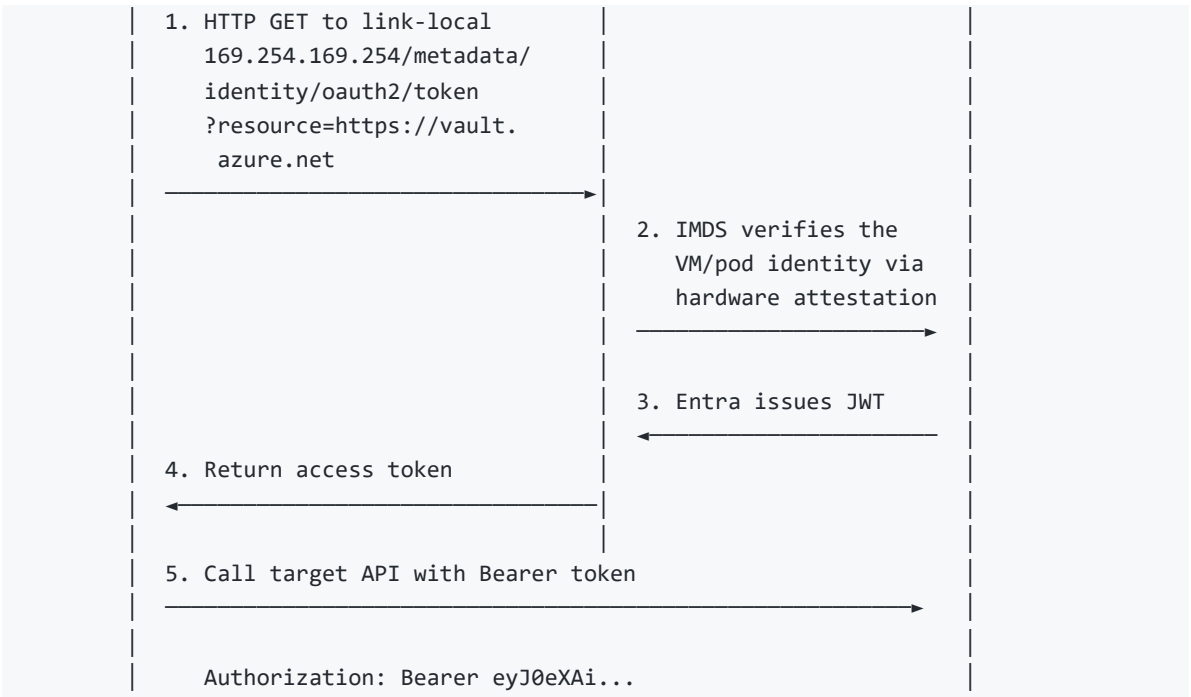
```
┌──────────────────────────────────────────────────────────────────┐
│              AGENT IDENTITY LANDSCAPE                            │
│                                                                  │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐ │
│  │  INSIDE AZURE    │  │  OUTSIDE AZURE   │  │  COPILOT STUDIO  │ │
│  │                  │  │                  │  │                  │ │
│  │  Managed Identity│  │  Service         │  │  Entra SSO       │ │
│  │  Entra Agent ID  │  │  Principal +     │  │  Manual OAuth    │ │
│  │  (zero secrets)  │  │  Client Creds    │  │  Maker creds     │ │
│  │                  │  │  WI Federation   │  │  (autonomous)    │ │
│  └──────────────────┘  │  mTLS / SPIFFE   │  └──────────────────┘ │
│                        └──────────────────┘                      │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │  MCP TOOLS: OAuth 2.1 + PKCE (remote) / No auth (local)  │    │
│  └──────────────────────────────────────────────────────────┘    │
│                                                                  │
│  ALL paths → issue OAuth 2.0 access tokens (JWT)                 │
│  ALL paths → governed by identity provider (Entra, Okta, etc.)   │
└──────────────────────────────────────────────────────────────────┘
```

Inside Azure: Managed Identity

Managed Identity is the **gold standard** for agents running inside Azure. The Azure infrastructure itself proves the agent's identity — **no secrets, no certificates, nothing to rotate**.

How It Works

```
Agent (in AKS / Functions / VM)        Azure IMDS              Microsoft Entra ID
       │                                  │                          │
```



Key insight: The agent NEVER sees or handles a secret.
The Azure fabric proves identity through infrastructure-level trust.

Two Flavors

Type	Lifecycle	Best For
System-assigned	Created with the resource, deleted with it. 1:1 relationship	Single-purpose agent on one compute resource
User-assigned	Standalone Entra object. Many resources can share it	Shared agent identity across multiple services, or agents that move between compute

Code Example (Python)

```python
from azure.identity import DefaultAzureCredential
from azure.keyvault.secrets import SecretClient

# Automatically uses Managed Identity when running in Azure
# Falls back to CLI/env credentials for local development
credential = DefaultAzureCredential()

client = SecretClient(
    vault_url="https://my-vault.vault.azure.net/",
    credential=credential
)
```

```
secret = client.get_secret("api-key")
# Agent never sees HOW it authenticated — the SDK handles it
```

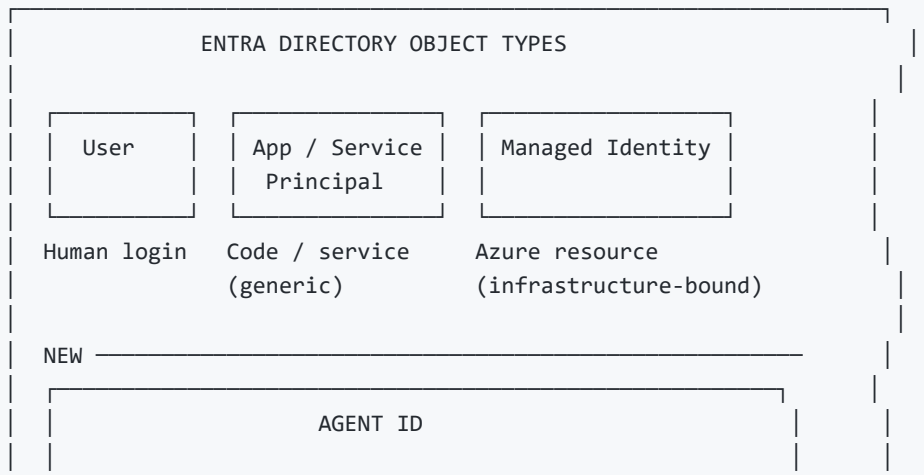
Inside Azure: Entra Agent ID

Entra Agent ID is Microsoft's new identity type purpose-built for AI agents. Unlike Managed Identity (infrastructure-bound) or Service Principals (generic), Agent ID is a **first-class Entra directory object specifically for agents**.

Why a New Identity Type?

Problem with Existing Types	How Agent ID Solves It
Service Principals are generic — no way to distinguish "agent" from "API app"	Agent ID is a **distinct object type** with metadata: purpose, owner, sensitivity
No agent-specific policies — Conditional Access treats agents like apps	Agent ID supports **per-agent Conditional Access** (e.g., "only allow from managed networks")
No agent lifecycle management — who owns this SP? When was it last used?	**Lifecycle tracking** — owner, creation source, last activity, expiry
Audit logs don't distinguish agent actions from app actions	**Per-agent audit trail** — every action traceable to a named agent
No governance at scale — 500 agents across an enterprise = chaos	**Centralized agent catalog** with inventory, classification, and policy enforcement

How It Differs



```
│  │  • Distinct object type — not user, app, or MI        │      │
│  │  • Owner + purpose metadata                           │      │
│  │  • Per-agent Conditional Access policies              │      │
│  │  • Full audit log (who did what, when)                │      │
│  │  • RBAC with least-privilege scoping                  │      │
│  │  • Auto-provisioned from Copilot Studio / Foundry     │      │
│  │  • Lifecycle management (create → monitor → retire)   │      │
│  └──────────────────────────────────────────────────────┘      │
└──────────────────────────────────────────────────────────────┘
```

Azure AI Foundry Identity Model (Deep Dive)

Motivation: Why Traditional Service Principals Fail for AI Agents

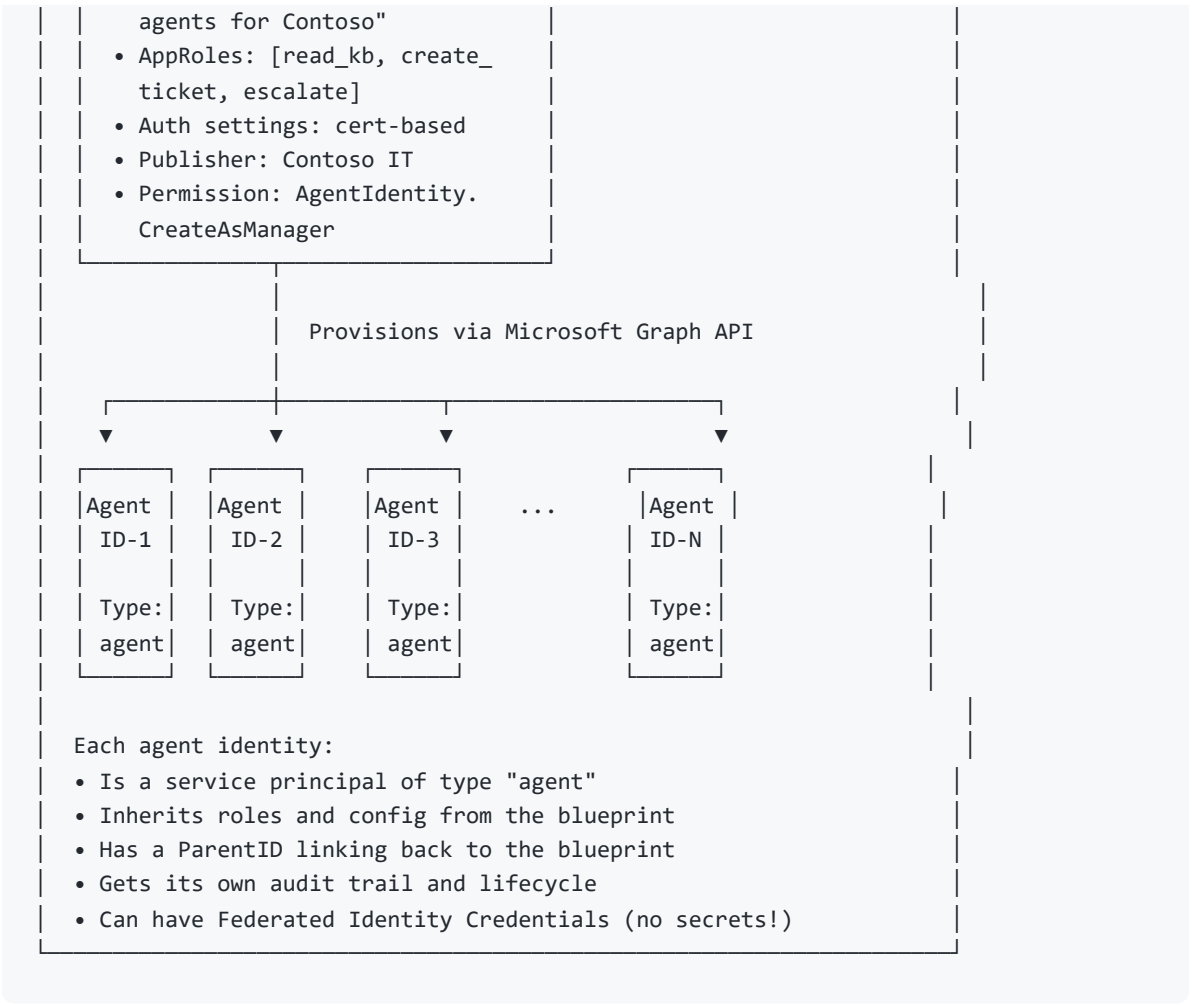
When organizations deploy hundreds of AI agents, traditional Service Principals break down:

```
┌──────────────────────────────────────────────────────────────┐
│  PROBLEM: Service Principals for AI Agents at Scale          │
│                                                              │
│  Developer creates Agent-A:                                  │
│    → Manually registers App in Entra                         │
│    → Creates a client secret (expires in 6 months)           │
│    → Stores secret in Key Vault (or worse, in code)          │
│    → Assigns RBAC roles manually                             │
│                                                              │
│  Multiply by 500 agents across 50 teams:                     │
│    → 500 secrets to rotate                                   │
│    → No way to tell which SP is an "agent" vs an "API"       │
│    → No centralized inventory — "agent sprawl"               │
│    → No agent-specific governance or Conditional Access      │
│    → SP deleted? No one knows which agent broke.             │
│                                                              │
│  Azure AI Foundry solves this with BLUEPRINTS.               │
└──────────────────────────────────────────────────────────────┘
```

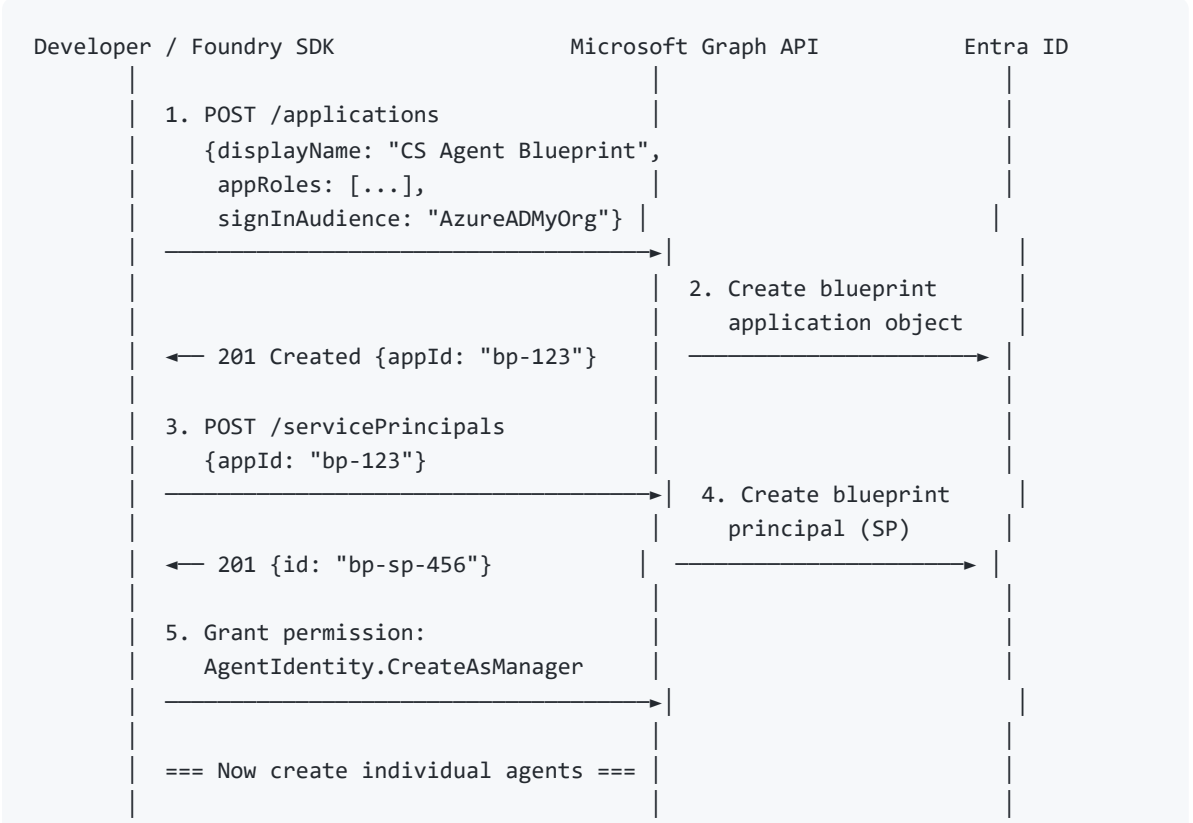
Agent Identity Blueprints

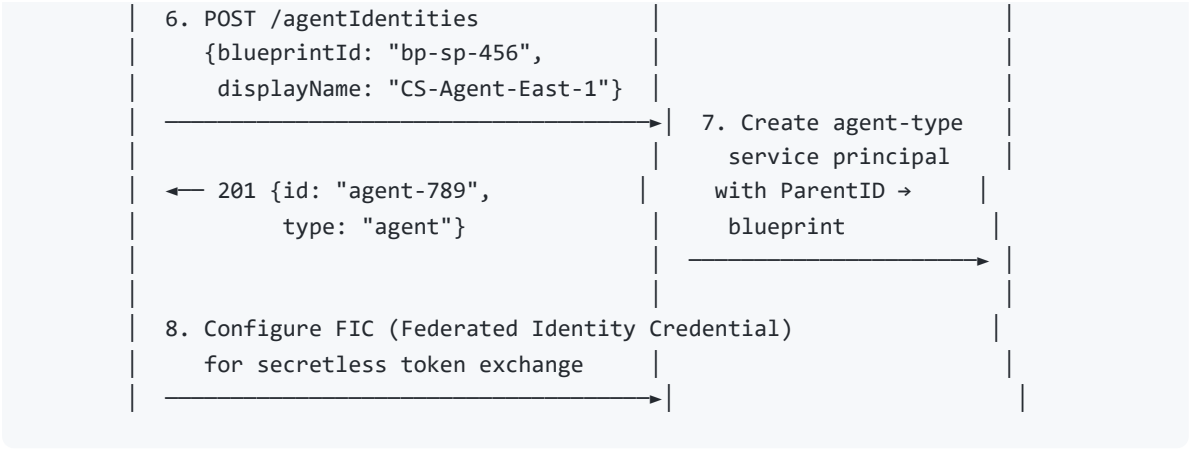
A **Blueprint** is a template object in Entra that defines how a class of agents should be configured. Think of it as a "factory" for agent identities.

```
┌──────────────────────────────────────────────────────────────┐
│              BLUEPRINT → AGENT IDENTITY PROVISIONING         │
│                                                              │
│  ┌──────────────────────────────────┐                        │
│  │     AGENT IDENTITY BLUEPRINT     │  (template in Entra)   │
│  │                                  │                        │
│  │  • Description: "Customer support │                       │
```



Provisioning Flow (Wire Level)



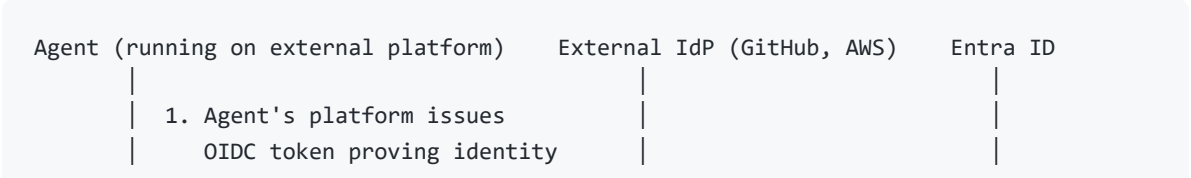


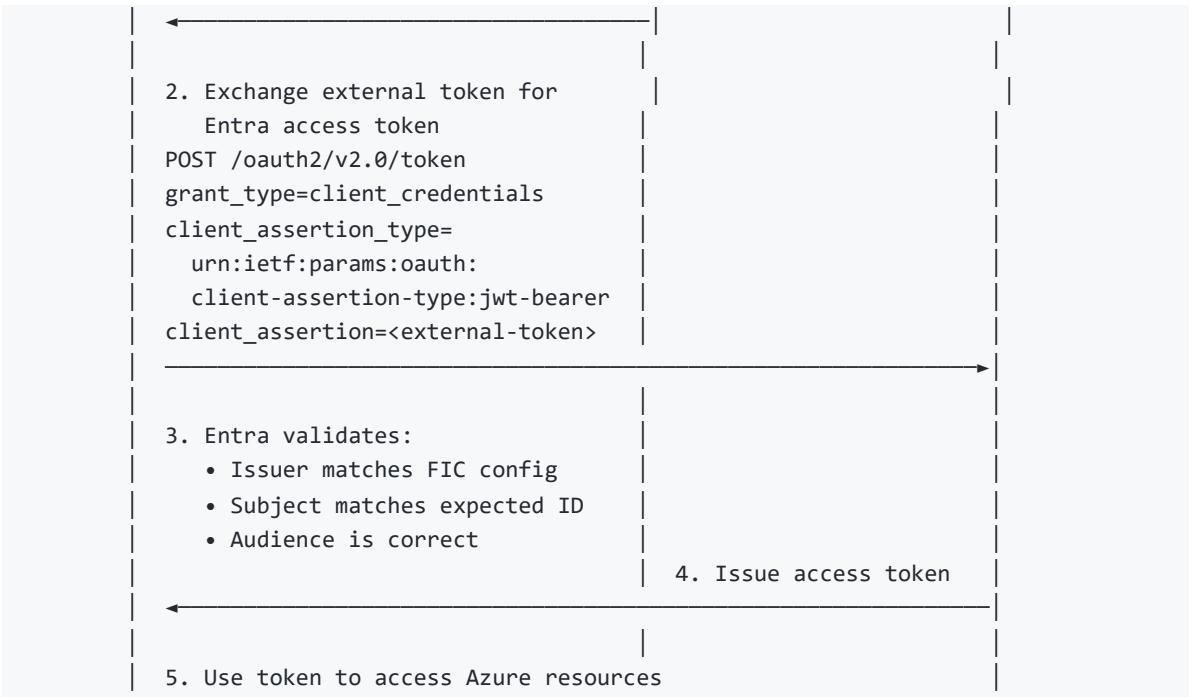
Blueprint vs Traditional Service Principal

Feature	Agent Identity Blueprint	Service Principal
Identity model	Hierarchical (blueprint → children)	Flat (one object per app)
Object type	`agent` type SP	Generic SP
Lifecycle management	Automated by Foundry	Manual scripts or ad-hoc
Credential handling	Blueprint holds creds; FIC for secretless	Each SP manages own creds
RBAC inheritance	Roles defined in blueprint, inherited by children	Per-SP manual assignment
Audit & inventory	Native via blueprint framework	Requires custom tracking
Scale	Designed for 100s-1000s of agents	Gets unwieldy beyond ~50
Governance	Conditional Access per agent, centralized catalog	Generic app policies only

Federated Identity Credentials (FIC)

FIC lets agent identities authenticate **without any secrets** by trusting tokens from an external identity provider:





Result: ZERO secrets stored anywhere. Trust is based on federation between the external IdP and Entra.

Outside Azure: Service Principals & Federation

For agents running **outside Azure** (on-prem, AWS, GCP), three options exist in increasing order of security:

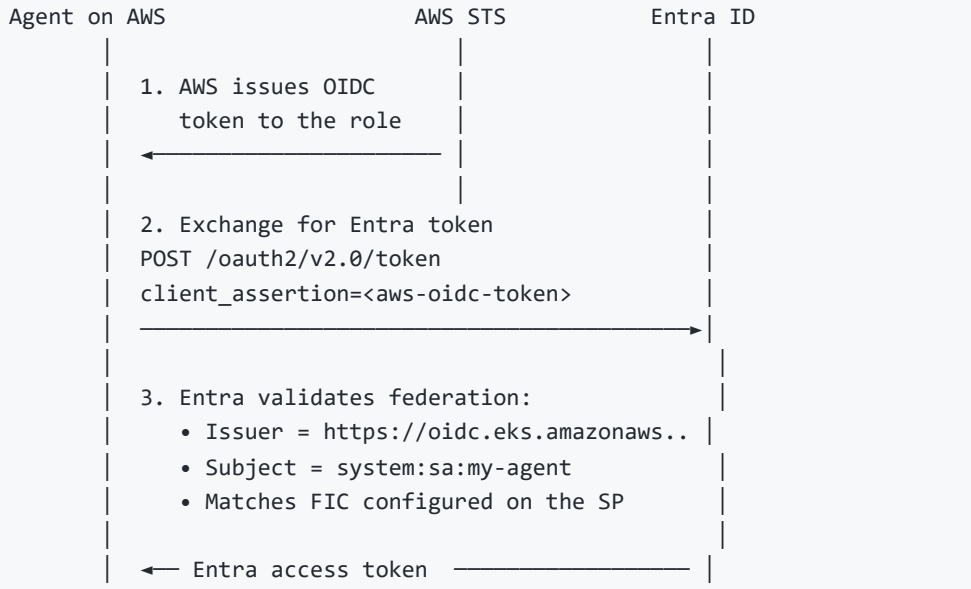
Option A: Client Secret (Simplest, Least Secure)



Option B: Certificate (More Secure)



Option C: Workload Identity Federation (Best — No Secrets)



Works for: AWS (EKS, Lambda), GCP (GKE), GitHub Actions, Kubernetes
Security: ZERO secrets. Trust based on platform identity federation.

Method	Secrets?	Security	Best For
Client Secret	✅ Yes (rotatable)	⚠️ Medium	Dev/test, quick PoC
Certificate	🔑 Private key (never transmitted)	✅ High	Production external agents

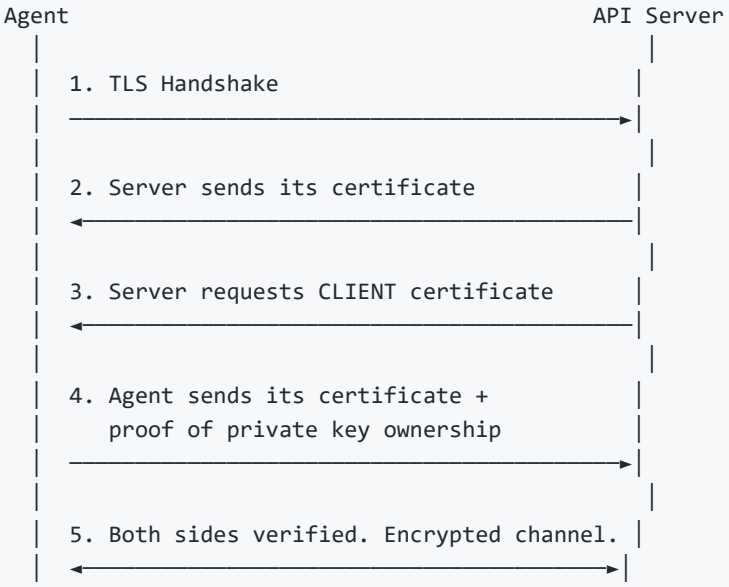
Workload ID Federation	❌ None	✅✅ Highest	Agents on AWS/GCP/GitHub Actions

Outside Azure: Platform-Agnostic Standards

Not everything is Azure. Here are identity standards used across the industry:

mTLS (Mutual TLS)

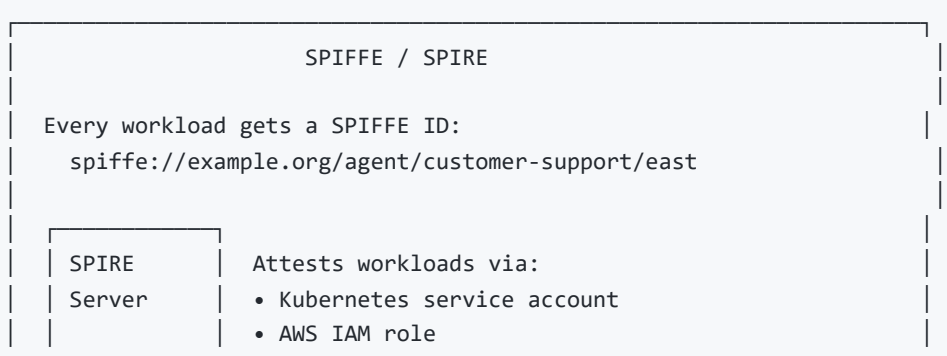
Both client AND server present certificates. The agent proves identity via its client certificate.



Used by: Service meshes (Istio, Linkerd), internal APIs, banking/financial systems, IoT device authentication

SPIFFE / SPIRE

SPIFFE (Secure Production Identity Framework for Everyone) is a CNCF standard for workload identity. **SPIRE** is its production implementation.



```
│  │            │  • Process UID/GID                                          │
│  └─────┬──────┘  • Docker labels                                           │
│        │                                                                    │
│    Issues SVIDs (SPIFFE Verifiable Identity Documents):                     │
│    • X.509-SVID — certificate for mTLS                                      │
│    • JWT-SVID — token for API auth                                          │
│        │                                                                    │
│    ┌───▼────┐  ┌──────────┐  ┌──────────┐                                   │
│    │ Agent-1 │  │ Agent-2 │  │ Agent-3 │  Each gets unique SVID          │
│    │ (SVID)  │  │ (SVID)  │  │ (SVID)  │  rotated automatically          │
│    └─────────┘  └─────────┘  └─────────┘                                    │
│                                                                             │
│  Why SPIFFE for agents:                                                     │
│  • Platform-agnostic (works on AWS, GCP, Azure, on-prem)                    │
│  • No static secrets — SVIDs auto-rotate                                    │
│  • Integrates with service meshes for zero-trust networking                 │
│  • Federable with Entra, Okta, etc. via OIDC                                │
└─────────────────────────────────────────────────────────────────────────────┘
```

API Keys (Simplest, Weakest)

```
Agent ──► API Server
  │
  │  Authorization: Bearer sk-abc123...
  │  X-API-Key: sk-abc123...
  │
  │  Problems:
  │  • Static secret — doesn't expire unless rotated manually
  │  • No identity information — just "someone has this key"
  │  • No scope — key either works or doesn't
  │  • No revocation granularity — rotate = break everything using it
  │
  │  Use only for: Low-sensitivity external APIs, rapid prototyping
```

Copilot Studio Authentication

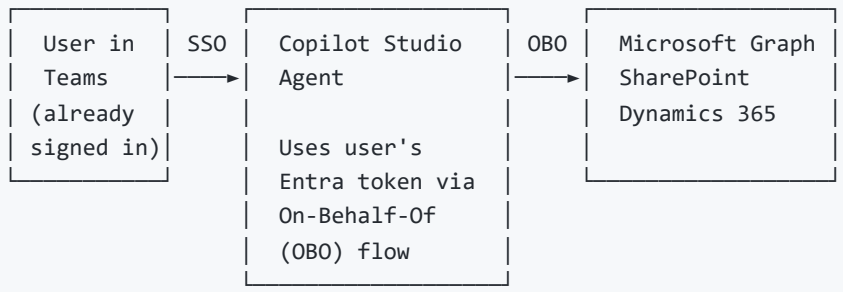
Copilot Studio agents support three authentication modes:

Mode 1: No Authentication

```
User ──► Copilot Agent ──► Public APIs only
(anyone can chat, no identity, no token)

Use for: Public FAQ bots, marketing chatbots
Risk: Agent can only access public data
```

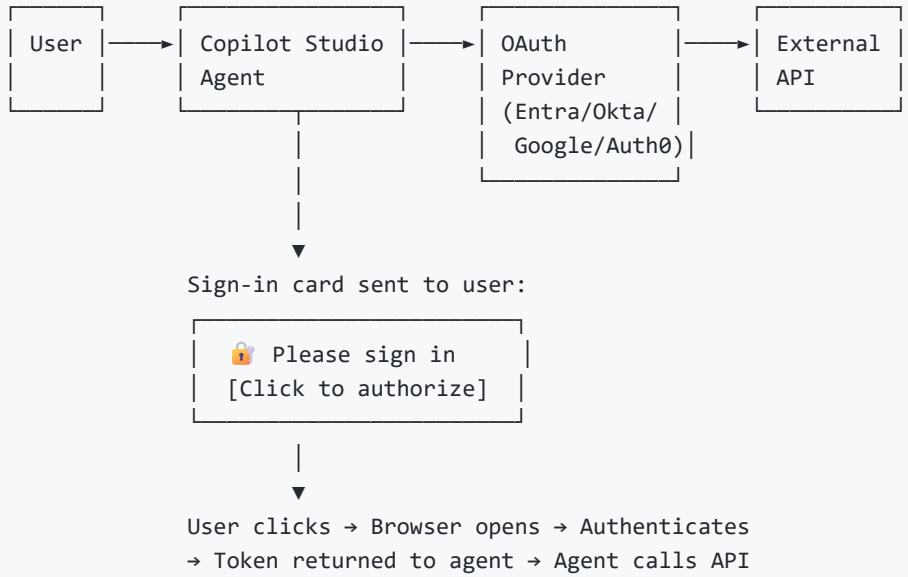
Mode 2: Entra ID SSO (Teams / M365 Only)



How it works:
1. User is already authenticated in Teams/M365
2. Teams passes the user's token to the agent
3. Agent uses OBO flow to exchange for a downstream token
4. Agent acts AS the user — inherits user's permissions
5. No extra login prompt (seamless SSO)

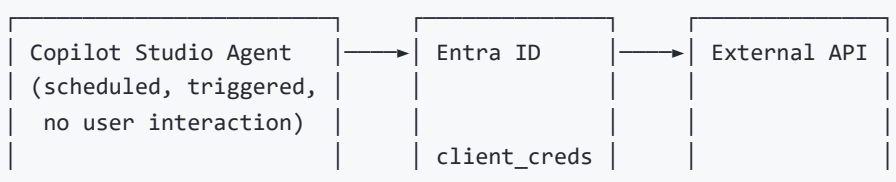
Limitation: Only works in Teams/M365 channels

Mode 3: Manual OAuth 2.0 (Any Channel, Any Provider)



Supports: Any OAuth2 provider, any channel (web, mobile, etc.)

Mode 4: Autonomous Agent (No User Present)



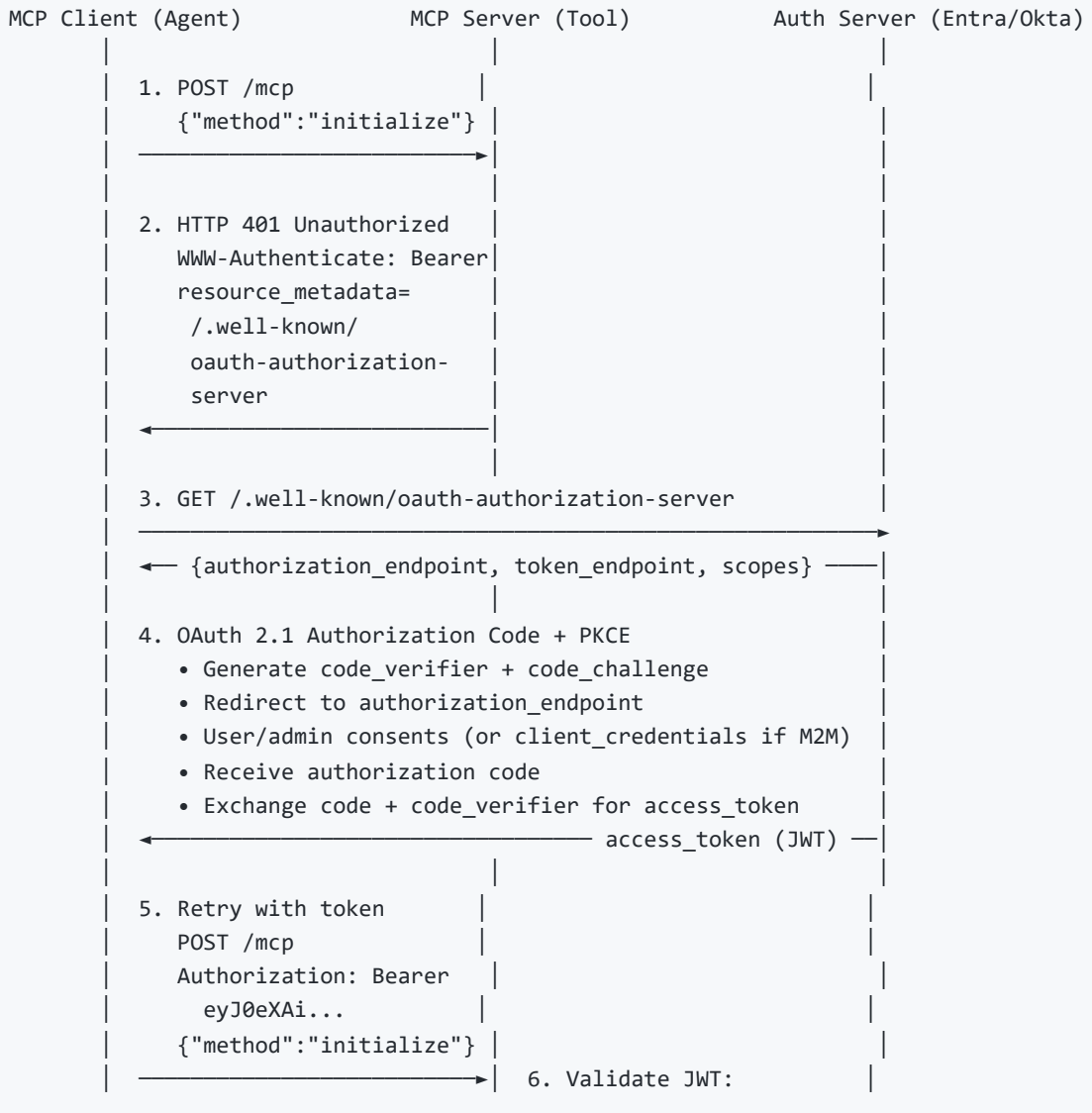
```
│ Uses MAKER's pre-      │     │ grant (no    │     │              │
│ configured credentials │     │ user consent)│     │              │
└────────────────────────┘     └──────────────┘     └──────────────┘

Security requirements:
• Maker configures credentials at design time
• Uses client_credentials grant (machine-to-machine, no interactive login)
• Credentials stored securely (Azure Key Vault recommended)
• Must follow least-privilege — only permissions the agent truly needs
• Regular rotation and monitoring required
```

MCP Tool Authentication

When an agent calls a **remote MCP server** (not local STDIO), authentication is required. MCP uses OAuth 2.1 with PKCE.

Wire-Level Flow





Local MCP (STDIO): No Auth Needed

```
For local MCP servers spawned as child processes:
• No network boundary → no authentication needed
• The process runs with the same permissions as the host
• Security comes from OS-level process isolation
• This is how most IDE-integrated MCP tools work (e.g., GitHub MCP in VS Code)
```

Identity Proof Matrix

How does each agent type prove "I am who I say I am"?

Scenario	Where Agent Runs	Identity Proof	Secret Required?	Wire Mechanism
Azure Managed Identity	Azure VM/AKS/Functions	Hardware attestation via IMDS	❌ None	GET 169.254.169.254 → JWT
Entra Agent ID	Azure AI Foundry	Blueprint-provisioned SP + FIC	❌ None (FIC)	Token exchange via Graph API
Service Principal (secret)	Anywhere	Client ID + secret	✅ Shared secret	POST /token + client_secret
Service Principal	Anywhere	Signed JWT assertion	🔑 Private key	POST /token + client_assertion

(cert)				
Workload ID Federation	AWS/GCP/GitHub	Platform OIDC token exchanged	❌ None	Platform token → Entra token
Copilot Studio SSO	Teams/M365	User's existing Entra session	❌ None (SSO)	OBO flow with user token
Copilot Studio OAuth	Any channel	User interactive login	✅ OAuth client secret	Auth code + PKCE
Copilot Studio Autonomous	Background (no user)	Maker's pre-configured creds	✅ Client creds	client_credentials grant
MCP (remote)	Network	OAuth 2.1 access token	Depends on flow	Bearer token in HTTP header
MCP (local/STDIO)	Same machine	OS process identity	❌ None	stdin/stdout (no network)
mTLS	Any network	Client X.509 certificate	🔑 Private key	TLS handshake with client cert
SPIFFE/SPIRE	Any platform	SVID (auto-rotated cert or JWT)	❌ None (auto)	mTLS or JWT based on SVID
API Key	Any	Static key string	✅ Static key	HTTP header

Security Ranking

```
Most Secure (no secrets, auto-rotating)
  ▲
  │  Managed Identity (hardware attestation, zero secrets)
  │  SPIFFE/SPIRE (auto-rotating SVIDs, platform attestation)
  │  Workload ID Federation (platform token exchange, zero secrets)
  │  Entra Agent ID + FIC (blueprint-managed, zero secrets)
  │  Certificate-based SP (private key never leaves machine)
  │  OAuth 2.1 + PKCE (short-lived tokens, no secret in URL)
  │  Client Secret SP (rotatable but leakable)
```

```
  │  Copilot Autonomous (maker credentials, must manage rotation)
  │  API Key (static, no expiry, no scope, no identity)
  ▼
Least Secure (static secrets, no rotation)
```

Interview Angle

Key Points to Articulate

1. **"Agents are non-human identities"** — they need the same governance as human users: least privilege, audit trails, lifecycle management, conditional access
2. **"No secrets is the goal"** — Managed Identity, Workload ID Federation, and SPIFFE eliminate secrets entirely. If you're storing a client_secret, you have room to improve
3. **"Azure AI Foundry uses Blueprints"** — not just Service Principals. Blueprints are templates that auto-provision agent-type SPs with inherited RBAC, FIC, and lifecycle tracking
4. **"MCP auth follows OAuth 2.1"** — remote MCP servers return 401 → client discovers auth metadata → gets token → retries. Local STDIO servers need no auth
5. **"Identity is the new perimeter"** — as CrowdStrike's 2025 GTR shows, 79% of attacks are malware-free and identity-based. Agent identity is the new attack surface

Common Interview Questions

Question	Key Talking Points
"How would you secure an AI agent accessing customer data?"	Managed Identity (Azure) or SPIFFE (multi-cloud), least-privilege RBAC, per-agent audit trail, Conditional Access
"What's the difference between MI and Service Principal?"	MI = infrastructure-bound, zero secrets; SP = portable, needs secret/cert. Use MI when possible
"How do you handle agent auth across clouds?"	Workload Identity Federation: AWS/GCP OIDC → Entra token exchange. Or SPIFFE for fully cloud-agnostic
"What is Entra Agent ID?"	First-class Entra object type for AI agents. Purpose metadata, per-agent policies, lifecycle management, distinct from users/apps
"How does Copilot Studio handle auth?"	3 modes: Entra SSO (Teams), Manual OAuth (any channel), Maker credentials (autonomous). Choice depends on

	channel + user presence

Further Reading

- Manage agent identities with Entra ID — Microsoft Docs
- Agent Identity Blueprints — Microsoft Entra
- Agent identities and service principals — Microsoft Entra
- MCP Authorization Specification (2025)
- Set Up MCP Server Authentication — Azure Foundry
- SPIFFE — Secure Production Identity Framework
- Copilot Studio Authentication — Microsoft Docs
- CrowdStrike 2025 Identity Threat Landscape